



Experiment 07: Deploying and Managing Containerized Applications using Kubernetes

Experiment No.	07
Mapped CO	CO4
Aim	To deploy and manage containerized applications using Kubernetes.
Tools / Software	Minikube or kind local cluster; kubectl; Docker Engine; A container image from earlier Docker experiment

Theory

Kubernetes is a container orchestration platform used to deploy, expose, scale and manage containerized applications across a cluster. It automates many operational tasks such as scheduling containers, maintaining the desired number of running instances and restarting failed workloads.

The smallest deployable unit in Kubernetes is a Pod. A Deployment manages Pods through ReplicaSets and keeps the desired number of replicas running. A Service provides a stable network endpoint for accessing the Pods, even when Pods are recreated or scaled.

Kubernetes follows a declarative model. The user writes YAML manifest files describing the desired state, such as number of replicas, container image, exposed port and service type. Kubernetes controllers continuously compare actual state with desired state and reconcile the cluster accordingly. This makes applications more reliable, scalable and manageable.

Procedure

1. Install Docker, kubectl and Minikube or kind on the local machine.
2. Start the Kubernetes cluster using minikube start or kind create cluster.
3. Verify the cluster using kubectl get nodes.
4. Prepare or pull a container image created in the Docker experiment, for example <user>/myapp:1.0.
5. Create a deployment.yaml file specifying application name, container image, container port and replica count.
6. Create a service.yaml file of type NodePort to expose the application outside the cluster.
7. Apply both YAML files using kubectl apply -f deployment.yaml -f service.yaml.
8. Check Pods, Deployments and Services using kubectl get pods, kubectl get deployments and kubectl get svc.
9. Access the application using minikube service myapp-svc --url or through the NodePort URL.
10. Scale the deployment using kubectl scale deployment myapp --replicas=4 and observe the new Pods.
11. Delete one Pod manually and verify Kubernetes self-healing by checking that a replacement Pod is created.
12. Clean up resources using kubectl delete -f deployment.yaml -f service.yaml and stop the local cluster.

Sample Code / Commands

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
```



```
spec:
  containers:
  - name: myapp
    image: <user>/myapp:1.0
  ports:
  - containerPort: 5000
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-svc
spec:
  type: NodePort
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 5000
```

Kubernetes commands

```
minikube start
kubectl get nodes
kubectl apply -f deployment.yaml -f service.yaml
kubectl get pods
kubectl get deployments
kubectl get svc
minikube service myapp-svc --url
kubectl scale deployment myapp --replicas=4
kubectl get pods
kubectl delete pod <pod-name>
kubectl get pods
kubectl delete -f deployment.yaml -f service.yaml
minikube stop
```

Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing Kubernetes cluster setup, application deployment, service exposure, scaling and self-healing verification. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Pushkar SHinde	58	B3	

1. Kubernetes Cluster Setup Output

Paste screenshot/output showing minikube/kind start command and kubectl get nodes output.

```
Pushkar@LAPTOP-0413C04Q MINGW64 ~/ps/Time pass/DevOps-APSIT (main)
$ minikube image load devops-apsit:2.0

Pushkar@LAPTOP-0413C04Q MINGW64 ~/ps/Time pass/DevOps-APSIT (main)
$ minikube image ls | grep devops-apsit
docker.io/library/devops-apsit:2.0
```

2. Deployment Manifest Output

Paste screenshot/output showing deployment.yaml with image name, replicas, labels and container port.

```
Pushkar@LAPTOP-0413C04Q MINGW64 ~/ps/Time pass/DevOps-APSIT (main)
$ minikube image load devops-apsit:2.0

Pushkar@LAPTOP-0413C04Q MINGW64 ~/ps/Time pass/DevOps-APSIT (main)
$ minikube image ls | grep devops-apsit
docker.io/library/devops-apsit:2.0
```

3. Service Manifest Output

Paste screenshot/output showing service.yaml with NodePort/LoadBalancer type, selector and target port.



```
Pushkar@LAPTOP-0413C04Q MINGW64 ~/ps/Time pass/DevOps-APSIT (main)
$ minikube service devops-apsit-service --url
http://127.0.0.1:52616
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

4. Apply and Resource Status Output

Paste screenshot/output showing kubectl apply and kubectl get pods, deployments and services output.

```
C:\Users\Pushkar\Downloads\APSIT\DEVOPS>kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE           NOMINATED NODE   READINESS
S GATES
devops-apsit-5cff689bb9-h9qrw      1/1     Running   0           101s  10.244.120.68   minikube       <none>           <none>
devops-apsit-5cff689bb9-mmqds      1/1     Running   0           101s  10.244.120.67   minikube       <none>           <none>
```

5. Application Access Output

Paste screenshot/output showing minikube service URL, browser output or curl response.

127.0.0.1:52616

Hello from DevOps APSIT - Docker v2!

6. Scaling and Self-Healing Output

Paste screenshot/output showing kubectl scale command, increased Pod count and replacement Pod after deletion.

```
C:\Users\Pushkar\Downloads\APSIT\DEVOPS>kubectl scale deployment devops-apsit --replicas=4
deployment.apps/devops-apsit scaled

C:\Users\Pushkar\Downloads\APSIT\DEVOPS>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
devops-apsit-5cff689bb9-h9qrw      1/1     Running   0           2m12s
devops-apsit-5cff689bb9-htpn9      1/1     Running   0            8s
devops-apsit-5cff689bb9-mmqds      1/1     Running   0           2m12s
devops-apsit-5cff689bb9-xgsqm      1/1     Running   0            8s
```

GitHub Repository / Lab Folder Link: <https://github.com/Pushkar3232/DevOps-APSIT>

Docker Hub / Container Image Link:

Final Observation Statement:

The containerized application was successfully deployed and managed using Kubernetes. The deployment was scaled from 2 to 4 replicas, and Kubernetes automatically recreated a deleted Pod, demonstrating self-healing

Expected Output

A local Kubernetes cluster should start successfully. The Deployment should create the requested number of Pods. The Service should expose the application through a stable endpoint. The application should be reachable through the Service URL. Scaling should increase the number of Pods, and self-healing should recreate a Pod if one is deleted.

Conclusion

A containerized application was deployed and managed on Kubernetes. The experiment demonstrated declarative deployment using YAML manifests, service exposure, scaling of replicas and Kubernetes self-healing behavior.

Viva / Review Questions



PARSHWANATH CHARITABLE TRUST'S

A.P. SHAH INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering
Data Science



-
1. What is a Pod and how does it differ from a container?
 2. What is the role of a ReplicaSet within a Deployment?
 3. Differentiate ClusterIP, NodePort and LoadBalancer Service types.
 4. How does Kubernetes achieve self-healing?

Marks: ____ / 10 **Date:** _____ **Signature of Faculty:** _____